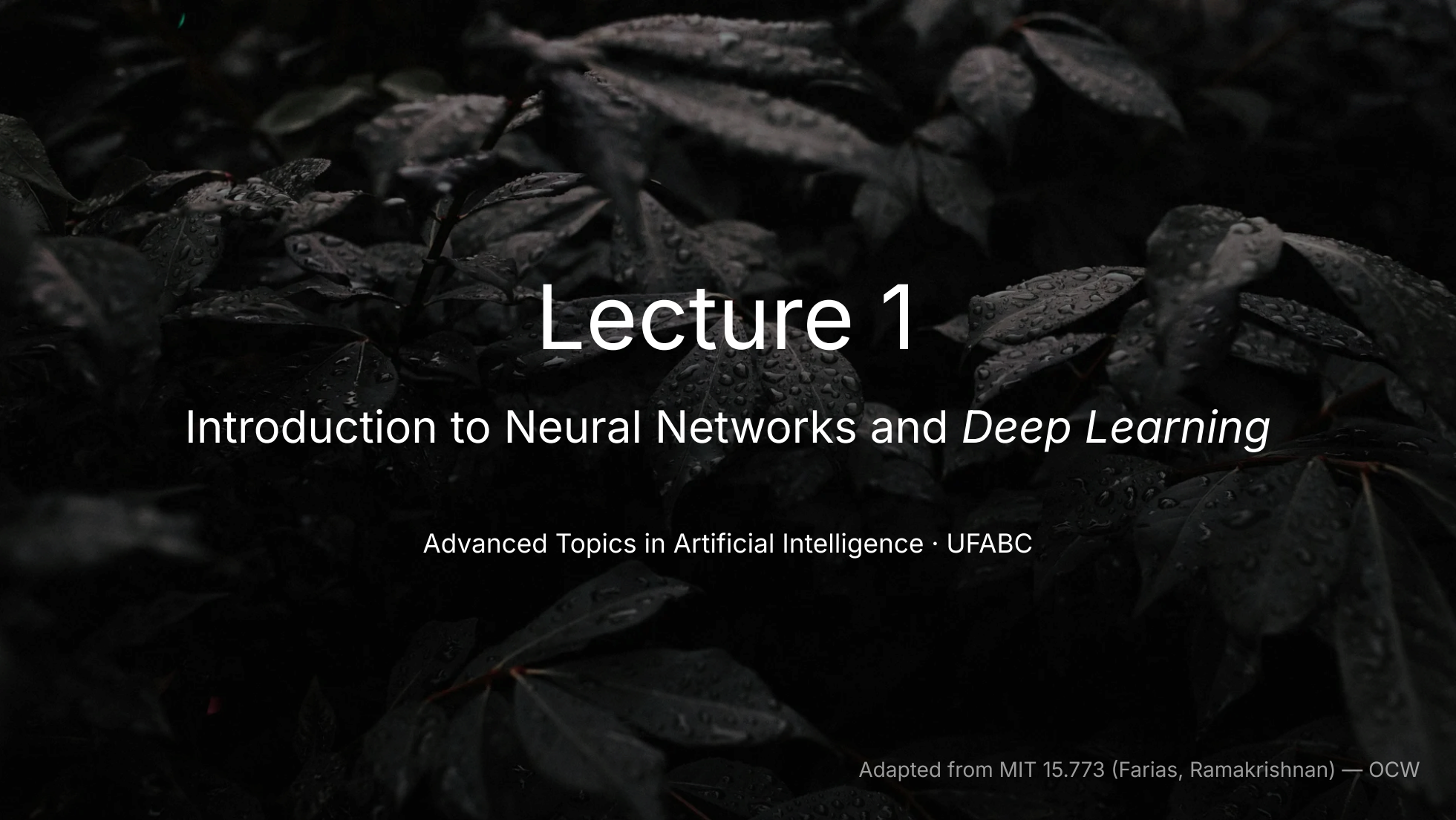


The background of the slide is a close-up photograph of dark green leaves, likely from a plant like a philodendron, which are covered in numerous water droplets. The lighting is low, creating a moody and textured appearance.

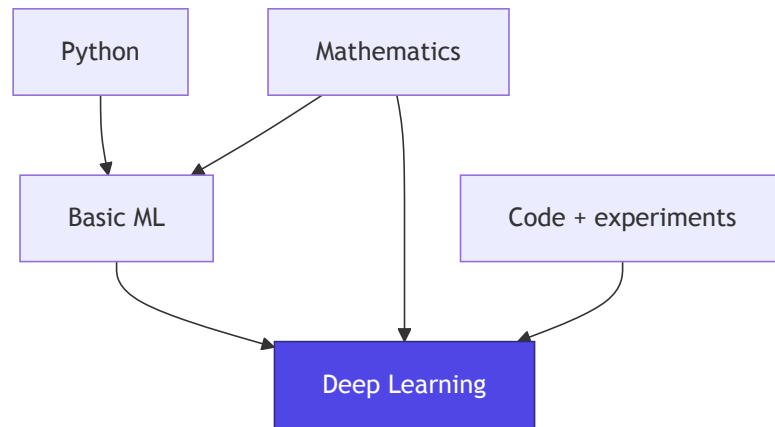
Lecture 1

Introduction to Neural Networks and *Deep Learning*

Advanced Topics in Artificial Intelligence · UFABC

Prerequisites

- Intermediate-level familiarity with **Python**
- Knowledge of **fundamental Machine Learning concepts**:
 - train / validation / test
 - *overfitting* and *underfitting*
 - regularization
- Good intuition with **linear algebra and calculus** (vectors, matrices, partial derivatives)
- Willingness to **write code** and experiment



Course philosophy

Concepts before mathematics · Hands-on before theory

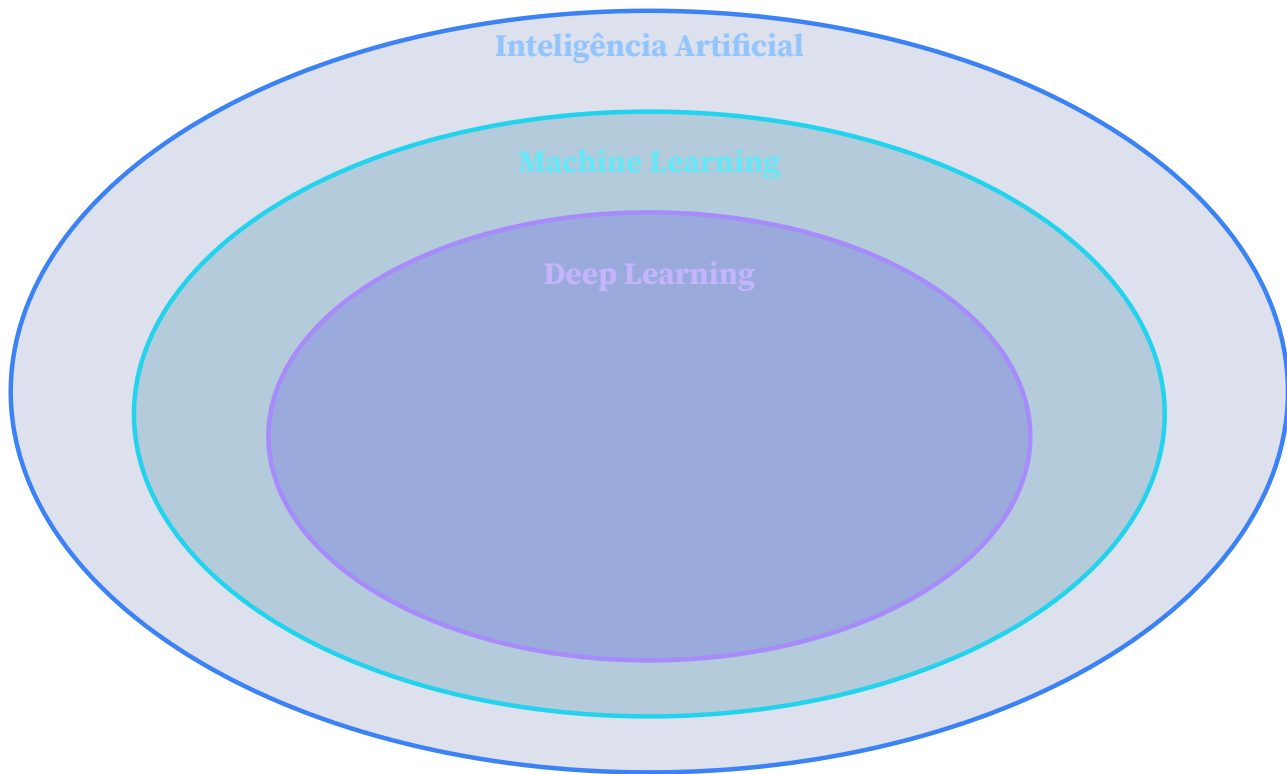
How we will approach the content

- **Focus on the key ideas** that underpin *Deep Learning*
- Mathematics appears **when it helps**, not as an obstacle
- Learning DL is like **learning to swim**: you can't do it just by watching
 - We will write, train, and debug real models
- The goal is not to train ML engineers, but to give you the autonomy to build a **V1.0 model** without depending on others

If you are looking for a strongly theoretical/mathematical approach, there are other more suitable courses.

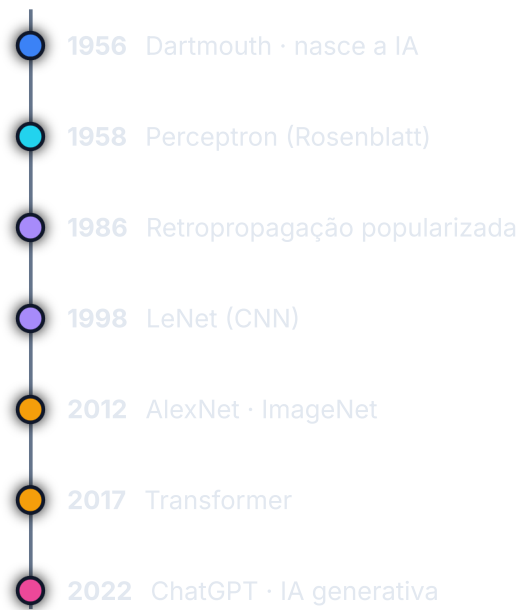
AI, ML, DL, and Generative AI

Before diving into neural networks, let's understand the relationships between these ideas.



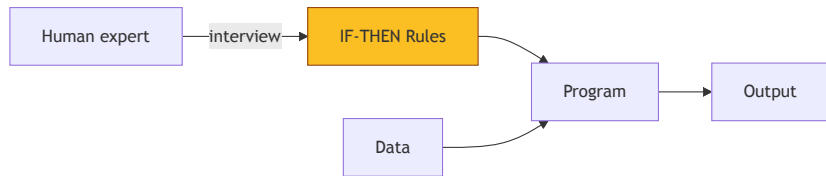
Artificial intelligence was born in 1956

- The term "**Artificial Intelligence**" was coined at a historic workshop held at **Dartmouth College** (summer of 1956)
- It brought together names such as **John McCarthy, Marvin Minsky, Claude Shannon, Allen Newell, Herbert Simon**
- It defined an optimistic agenda: machines that could learn, reason, and use language
- Since then, AI has gone through several **winters** and **revivals**



The classical AI approach

- **Goal:** give the computer the ability to perform tasks that only humans did well
- **Classical strategy:** ask *human experts* how they do it, transcribe into **IF...THEN rules**, program them explicitly
- Worked in **some well-delimited domains** (expert systems, rule-based chess)



Why is this so hard?

"We know more than we can tell." — Polanyi's Paradox

- We recognize a face, ride a bicycle, identify sarcasm — but it is **very hard to write the rules** that describe how we do it
- Explicit rules don't cover **edge cases** nor generalize to new situations
- Result: symbolic AI ran into the **complexity of the real world**

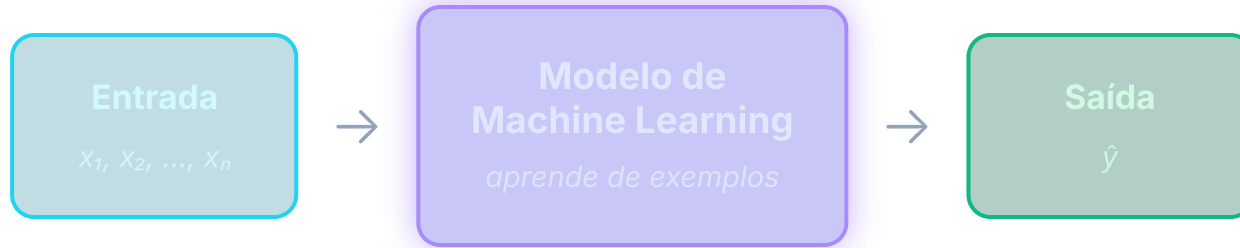
Paradigm shift

Instead of **telling** the computer what to do...

...**show** it *many examples* of input and output, and let **statistical techniques** learn the relationship.

This is Machine Learning

Machine Learning, in one figure



Classical algorithms: linear regression, logistic regression, trees, *random forests*, *gradient boosting*, SVMs, shallow neural networks...

ML shines with **structured data**

- Structured data = data that fits **naturally into a spreadsheet**
- Each column is a meaningful numerical/categorical *feature*
- Classical ML works **very well** in these cases:
 - credit score
 - demand forecasting
 - fraud detection
 - diagnosis based on laboratory tests

age	income	owns_property	defaulted
32	5.4k	yes	0
47	3.1k	no	1
25	2.8k	no	0
51	9.2k	yes	0

Illustrative example (synthetic)

But what about **unstructured data**?



Images

RGB pixels without isolated meaning



Text

sequences of characters/tokens



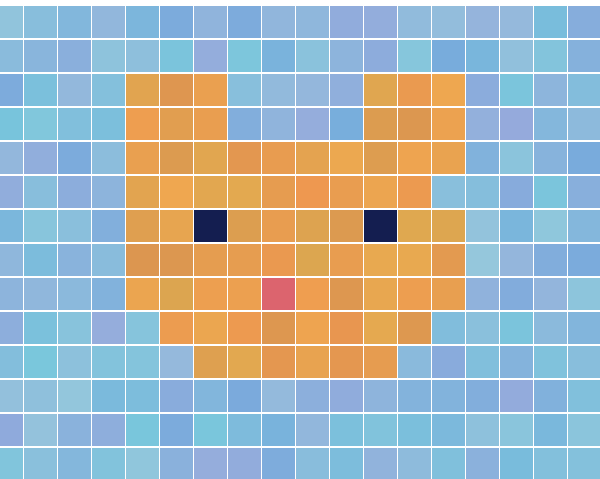
Audio

raw temporal samples

The **raw form** of this data has no intrinsic meaning for classical algorithms. A pixel (R=128, G=64, B=200) says nothing on its own about whether there is a cat in the image.

The difficulty of unstructured data

"O que vemos"



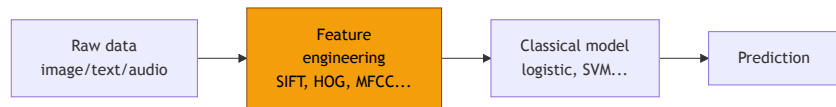
"O que o computador vê"

(145,196,220)	(136,190,220)	(130,183,220)	(146,183,220)	(124,182,220)	(124,171,220)
(138,187,220)	(137,181,220)	(138,175,220)	(142,195,220)	(142,191,220)	(123,196,220)
(125,171,220)	(123,192,220)	(147,184,220)	(132,192,220)	(225,164,80)	(222,150,80)
(120,196,220)	(129,199,220)	(129,191,220)	(124,191,220)	(238,158,80)	(225,158,80)
(147,183,220)	(145,174,220)	(124,171,220)	(140,189,220)	(233,160,80)	(220,155,80)
(145,173,220)	(136,190,220)	(140,173,220)	(141,180,220)	(226,164,80)	(239,167,80)
(123,183,220)	(136,197,220)	(138,191,220)	(130,175,220)	(223,159,80)	(231,165,80)
(143,183,220)	(124,188,220)	(137,179,220)	(137,188,220)	(220,150,80)	(220,151,80)

A classifier *does not* see the cat — it sees a matrix of numbers.

The pre-DL solution: **feature engineering**

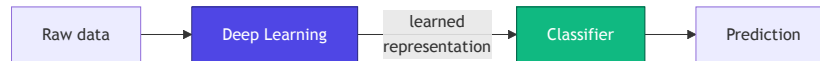
- Experts designed **handcrafted representations** of the data:
 - **SIFT, HOG, SURF** for images
 - **MFCC** for audio
 - **TF-IDF, *bag-of-words*** for text
- The extracted representation was then fed into a **classical** model (usually logistic regression!)



This required **a great deal of human effort** — a *bottleneck* that drastically limited the reach of ML on rich data such as images, speech, and natural language.

What does **DL** do that classical ML didn't?

- **Automatically extracts** useful representations from unstructured data
- These representations can feed even trivial models (a logistic regression on top already delivers impressive results)
- Solves the **human bottleneck** that limited ML on images, text, and audio



Feature engineering stops being *human* and becomes *learned*.

Why did it happen **now**?



Algorithms

ReLU, dropout, batch norm, convolutional and transformer architectures, optimizers (Adam, etc.)



Data

Digitization of everything: photos, videos, social networks, sensors, logs.
ImageNet, Common Crawl...



Compute

GPUs, then TPUs. Massive parallel training in reduced precision.

...applied to an **old** idea: **artificial neural networks**.

Immediate application: **perception**

Each **sensor** can gain the ability to detect, recognize, and classify what it is perceiving. *Coupling* DL to cameras, microphones, and sensors creates qualitatively different products.



Object detection



Medical imaging diagnosis



Autonomous driving



Industrial inspection



Speech recognition



Microscopy analysis



Remote sensing



Bio-acoustics

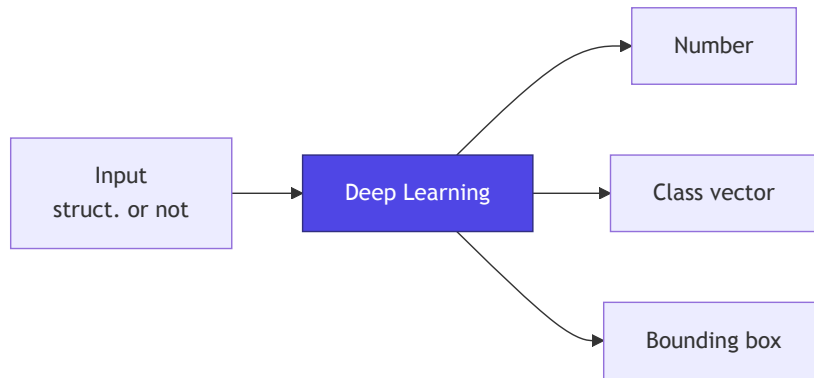
And what about **output**?

Classical DL easily predicted **structured outputs** (a number, a label, a probability vector).

But **generating** text, images, audio, code? For a long time, that was **difficult territory**.

Outputs that ML/DL predicted well

- **A number**
 - probability of default
 - next week's demand
- **A few numbers**
 - distribution over 1,000 ImageNet classes
 - coordinates of a *bounding box*
- **A discrete label**
 - sentiment (positive/negative)
 - product category

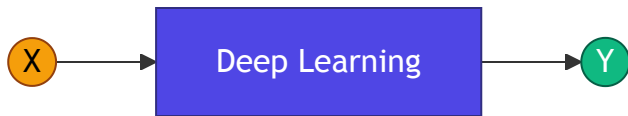


The $X \rightarrow Y$ "matrix" of Generative AI

Both X and Y can be text, image, audio, video, code... and even **multimodal**.

$X \setminus Y$		 Texto	 Imagem	 Áudio	 Código
 Texto		tradução, resumo, chat	geração de imagens	TTS, música	geração de código
 Imagem		legendas, OCR	estilização, edição	sonificação	sketch → app
 Áudio		transcrição	visualização	remoção de ruído	—
 Código		documentação	diagramas	síntese sonora	tradução de linguagens

Summary: $X \rightarrow Y$



All the current excitement about AI is, in essence, the success of **Deep Learning** (and the Generative AI that came on top of it).

What is a neural network?

Let's start from the beginning.

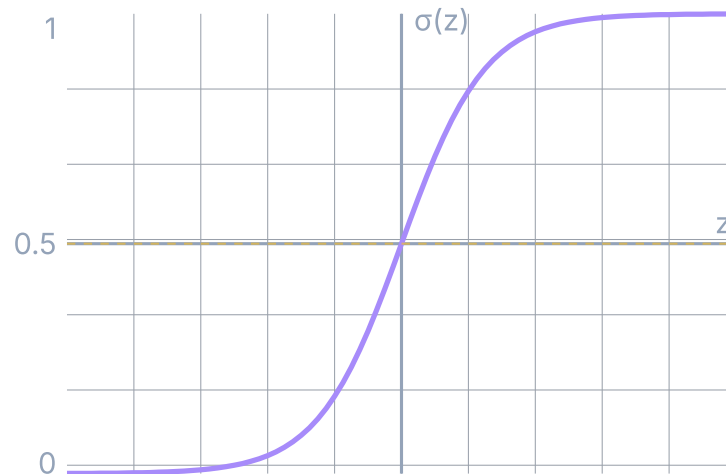
Back to logistic regression

Logistic regression maps an input vector to a **probability**:

$$p(y = 1 \mid \mathbf{x}) = \sigma \left(b + \sum_{i=1}^n w_i x_i \right)$$

where $\sigma(z) = \frac{1}{1 + e^{-z}}$ is the **sigmoid** function.

Let's look at this expression as a **network of mathematical operations**.



$$\sigma(z) = 1 / (1 + e^{-z}) \text{ — saída em } (0, 1)$$

Example: predicting who gets called for an interview

- **Inputs:**

- GPA (grade point average)
- Years of experience

- **Output:** probability of being called for an interview (1 = called, 0 = not called)

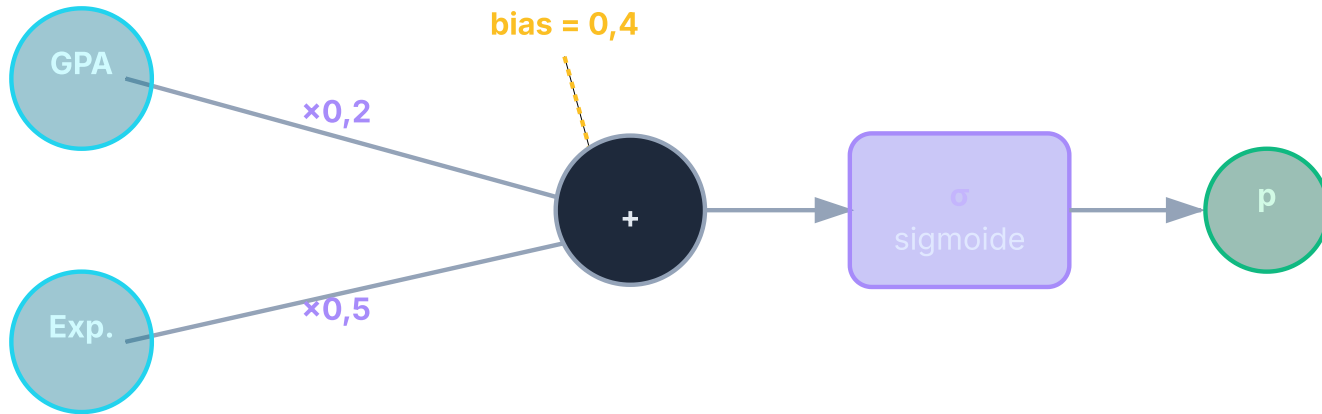
- **Model:** logistic regression trained on historical data

#	GPA	Exp.	Called
1	3.27	1.93	0
2	3.37	0.07	0
3	3.57	1.91	0
4	3.91	4.35	0
6	3.90	2.41	1
7	3.94	3.00	1
15	3.77	2.06	1

Suppose we trained and obtained:

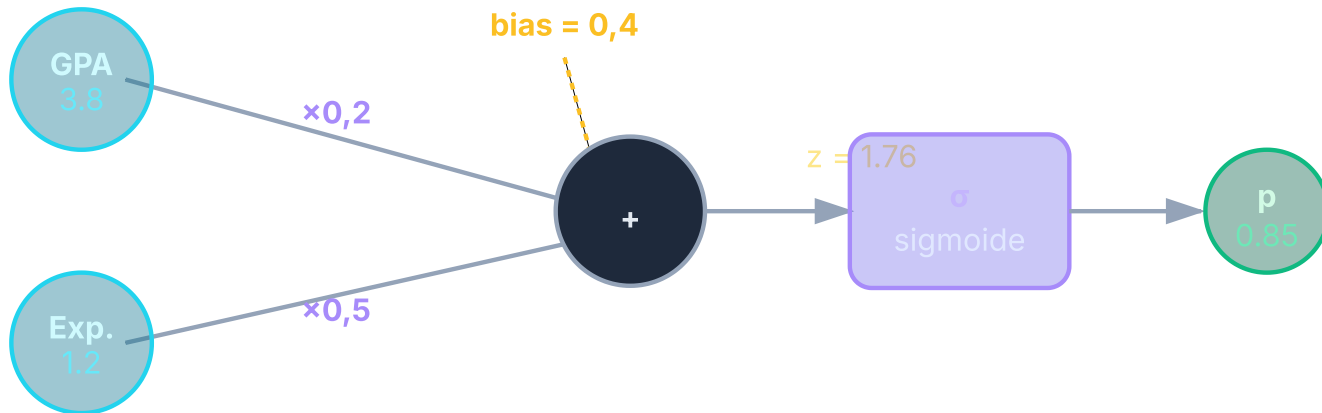
$$p(\text{called}) = \sigma(0,4 + 0,2 \cdot \text{GPA} + 0,5 \cdot \text{Exp})$$

Rewriting as a **network** with flow from left to right:



Predicting with the "network"

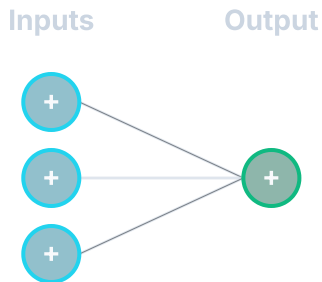
Consider a candidate with **GPA = 3.8** and **1.2 years** of experience.



$$\sigma(0,4 + 0,2 \cdot 3,8 + 0,5 \cdot 1,2) = \sigma(1,76) \approx 0,85$$

The estimated probability of being called for an interview is about **85%**.

Basic vocabulary



Weights — multipliers on the connections between neurons

Neuron — unit that computes $\sum + \text{bias}$ and applies activation

Bias — additive term (intercept) at each neuron

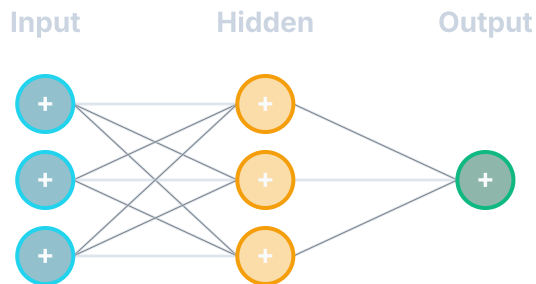
Layer — vertical stack of neurons

Why is the "network lens" useful?

- It allows us to **transform** the input data before the final decision
- Instead of feeding raw data directly to logistic regression, we can first **learn a better representation** of it
- This is the heart of *Deep Learning*: stacking learned transformations

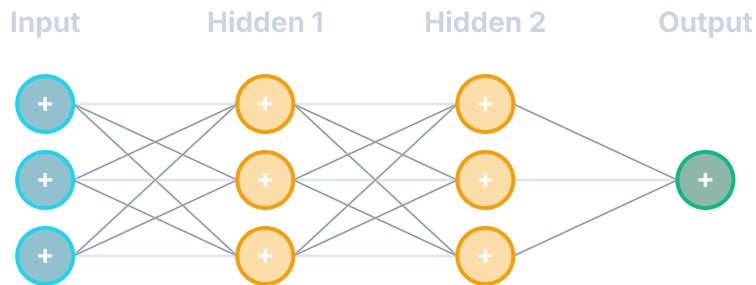
Stacking linear functions

Before the final decision, **insert** a layer of linear functions that combines the inputs:



Each node in the intermediate layer is $z_j = b_j + \sum_i w_{ji}x_i$ — a **linear transformation** of the inputs.

Stacking even more



We can stack **as many layers** as we want. At the end, we feed logistic regression (sigmoid) with the transformed vector.

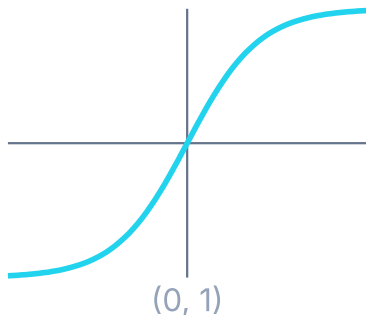
⚠ But with only *linear* layers the composition remains linear — we need something more.

Non-linear activation functions

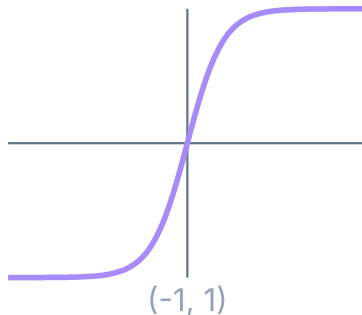
For stacking layers to make a difference, each neuron applies a **non-linear activation function** after the linear combination:

$$a_j = \phi\left(b_j + \sum_i w_{ji} x_i\right)$$

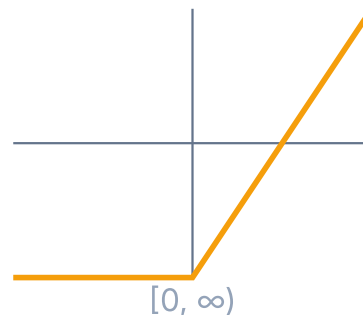
Sigmoide



Tanh



ReLU



Common activation functions

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Output in (0,1). Used in the **output layer** for binary classification. Suffers from *vanishing gradients* in deep layers.

Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Output in (-1,1), centered at zero. Better behaved than sigmoid, but still saturates.

ReLU

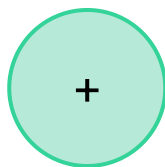
$$\text{ReLU}(z) = \max(0, z)$$

Modern standard in hidden layers. Cheap, avoids saturation for $z > 0$, enables much deeper networks.

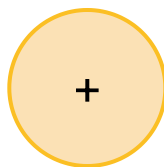
Others: Leaky ReLU, ELU, GELU, Swish, Softmax (in multi-class output)...

Visual notation

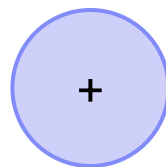
From here on, we will abbreviate each neuron with a **circle**, indicating the activation by color/label:



Linear

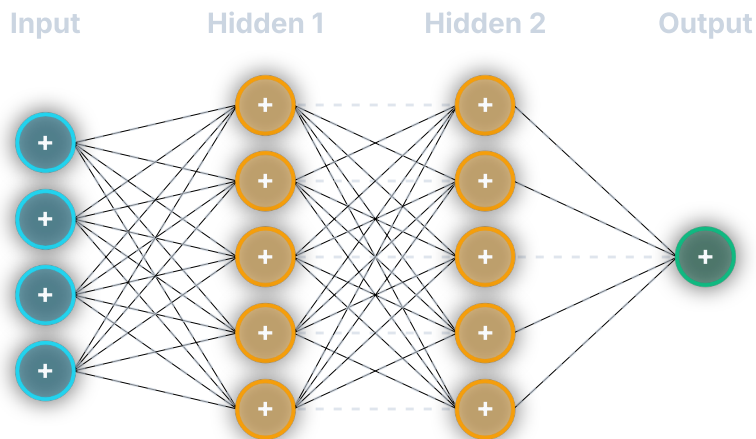


ReLU



Sigmoid

Anatomy of a DNN



Input layer: the variables $x_1, \dots, x_k$

Output layer: produces the prediction

Hidden layers: transform the representation

Dense connections: each neuron connects to all neurons in the next layer

When all layers are dense, we say the network is *fully connected*.

Designing a DNN: the choices

When you design a feedforward (*vanilla*) neural network, you decide:

How many hidden layers?

depth

How many neurons per layer?

width

Which activation in the hidden layers?

typically ReLU/GELU

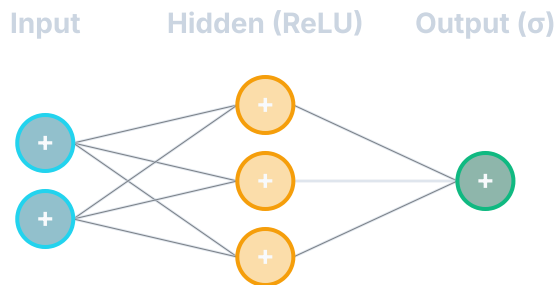
Which activation in the output?

depends on the task

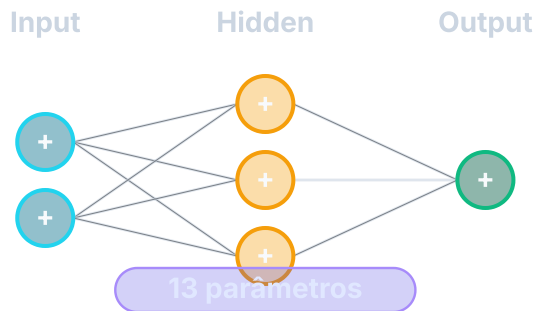
The choice of output activation is guided by the nature of y : linear (regression), sigmoid (binary classification), softmax (multi-class).

Applying to the interview classifier

- **Inputs:** 2 variables (GPA, experience)
- **Output:** probability $\in (0, 1)$
- **Design decisions:**
 - 1 hidden layer with **3 ReLU neurons**
 - **Sigmoid** at the output (probability)

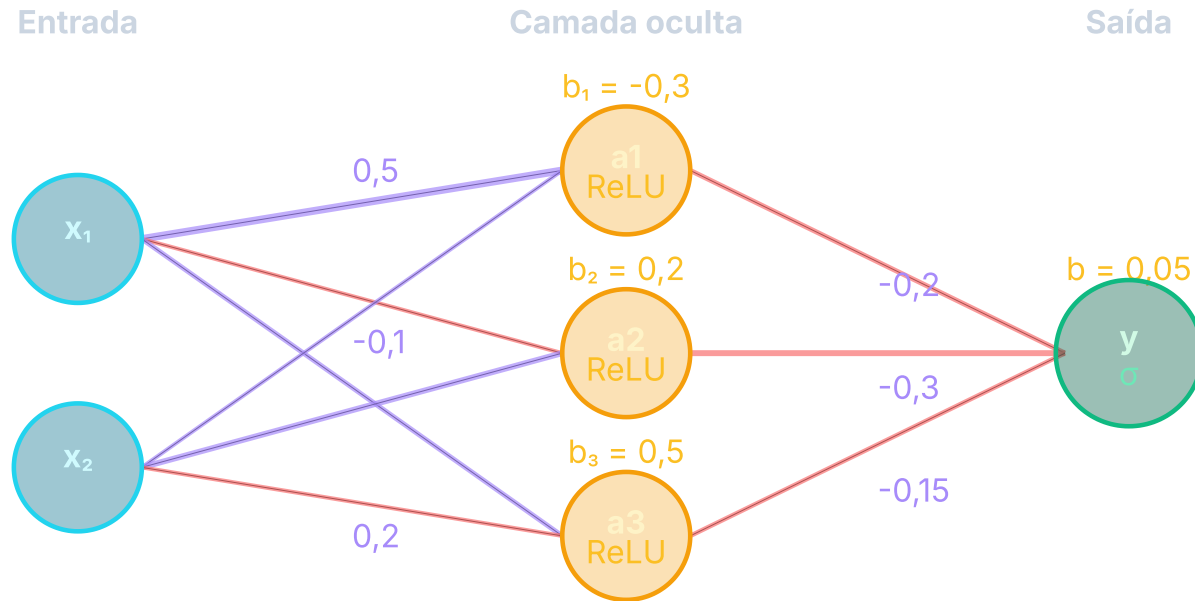


How many parameters does this network have?



- From input to hidden: $2 \times 3 = 6$ weights + 3 biases = 9
- From hidden to output: $3 \times 1 = 3$ weights + 1 bias = 4
- **Total: 13 parameters**

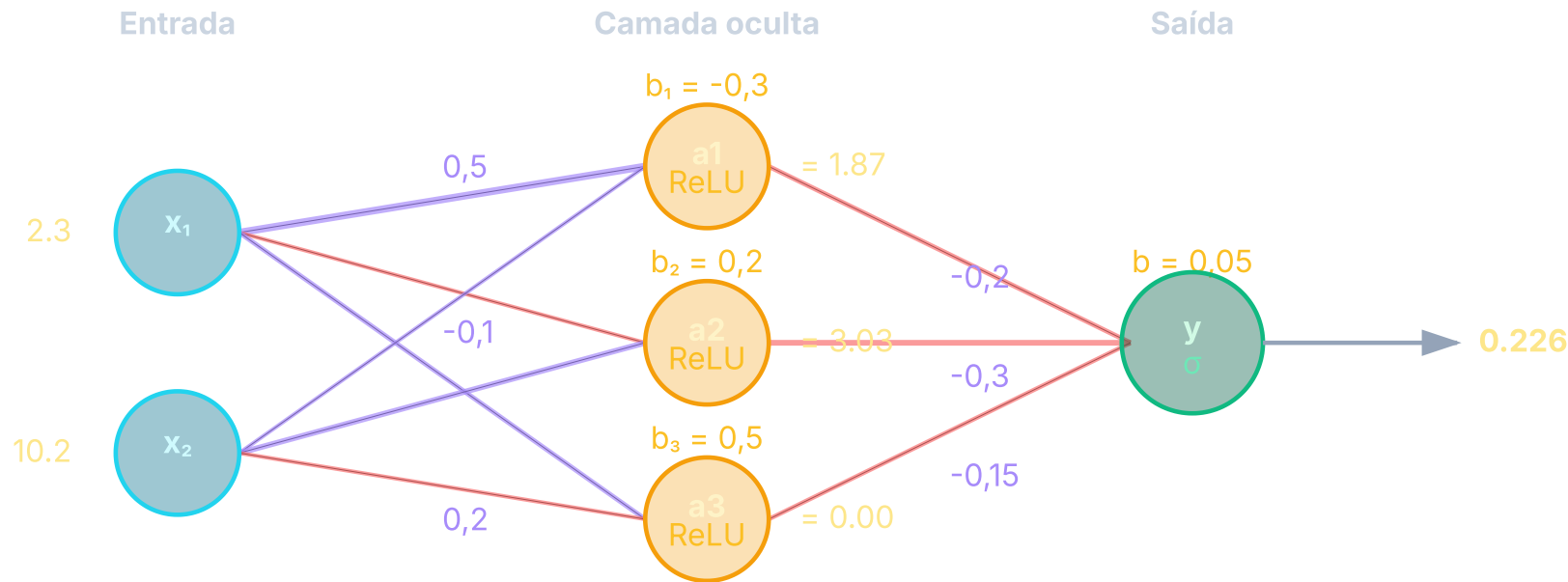
Trained weights (assumption)



How these weights are *found* from data is the topic of the **next lecture** (training, loss, backpropagation).

Forward pass — computing a prediction

For a candidate with $x_1 = 2,3$ (GPA) and $x_2 = 10,2$ (experience*):



Hidden layer: explicit calculations

$$a_1 = \text{ReLU}(-0,3 + 0,5 \cdot 2,3 + 0,1 \cdot 10,2) = \max(0; 1,87) = 1,87$$

$$a_2 = \text{ReLU}(0,2 - 0,1 \cdot 2,3 + 0,3 \cdot 10,2) = \max(0; 3,03) = 3,03$$

$$a_3 = \text{ReLU}(0,5 + 0,2 \cdot 2,3 - 0,1 \cdot 10,2) = \max(0; -0,06) = 0$$

And the output:

$$y = \sigma(0,05 + (-0,2) \cdot 1,87 + (-0,3) \cdot 3,03 + (-0,15) \cdot 0) \approx \sigma(-1,23) \approx 0,226$$

For this candidate, the network estimates $\approx$ **22.6%** chance of being called.

The network as a function

The network's prediction can be written as **a single function** of the inputs:

$$\hat{y}(\mathbf{x}) = \sigma\left(\mathbf{w}^{(2)\top} \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + b^{(2)}\right)$$

Pure logistic regression:

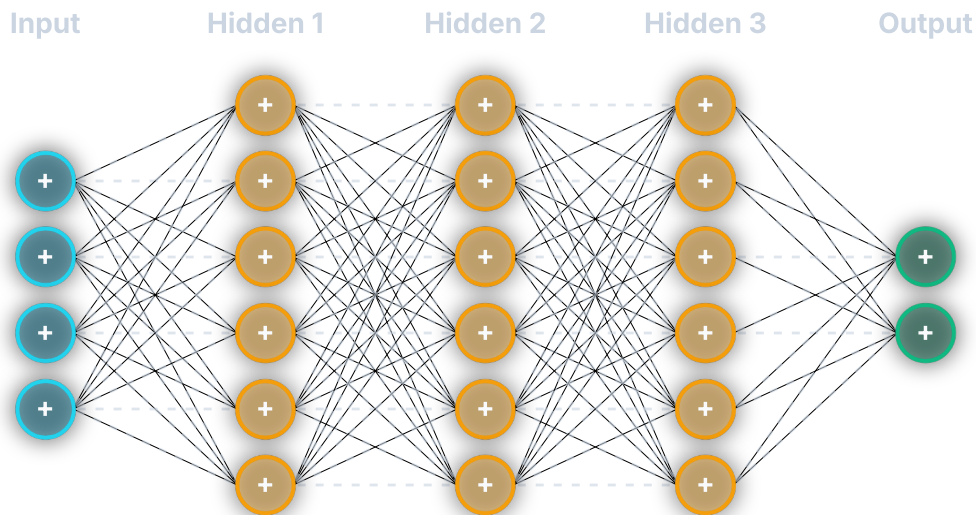
$$\hat{y} = \sigma(0,4 + 0,2x_1 + 0,5x_2)$$

linear in the data before the sigmoid.

Network with 1 hidden layer:

a much richer non-linear composition — capable of representing complex relationships between $\mathbf{x}$ and y .

Summary: a DNN



- **Feedforward** (vanilla): data flows from left to right
- **Architecture** = layers + activations + connections
- **Depth** comes from many hidden layers
- *Deep Learning* is, in essence, **neural networks with many layers**
- Training, backpropagation, and optimization — next lecture

Recap

Hierarchy of ideas

AI $\supset$ ML $\supset$ DL $\supset$ Generative AI

Why DL is special

learns representations from unstructured data

The combination that unlocked it

algorithms + data + GPUs

Neural network $\approx$ regression

logistic stacked with non-linearities

Components

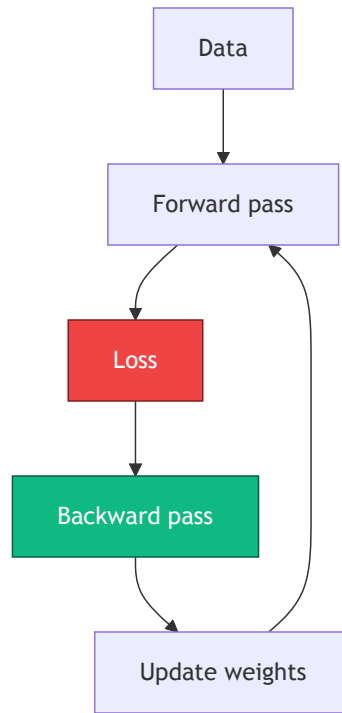
layers, neurons, weights, biases, activations

Forward pass

linear combination $\rightarrow$ non-linearity $\rightarrow$ repeat

Next lecture

- How do you **train** a neural network?
- What is a **loss function**?
- **Gradient descent** and its variants
- **Backpropagation** (intuition + a numerical example)



Thank you! Questions?

Freely adapted from *15.773 Hands-on Deep Learning* (MIT OpenCourseWare, 2024) — original English material by Vivek Farias and Rama Ramakrishnan, distributed under the terms of MIT OCW.

For more information: <https://ocw.mit.edu/terms>